

Polygon Middleman

Developer Documentation

A full-featured desktop web application for managing
Codeforces Polygon problems via the Polygon API

Version 1.0 — April 2026

Stack: Python FastAPI + React 18 + TypeScript + Tailwind CSS

Table of Contents

1. Project Overview
2. Architecture
3. Getting Started
4. Backend Deep Dive
5. Frontend Deep Dive
6. Upload Wizard (8-Step Flow)
7. Polygon API Coverage
8. Key Features & Implementation Notes
9. Theme & Design System
10. File Structure Reference

1. Project Overview

Polygon Middleman is a desktop web application that provides a modern, user-friendly interface for uploading and managing competitive programming problems on Codeforces Polygon. Instead of manually navigating Polygon's web UI for each field, this tool lets you manage everything from a single dashboard — statements, tests, solutions, checkers, validators, packages, and more.

The app runs locally: a Python FastAPI backend acts as a secure proxy that handles Polygon's SHA-512 API authentication, while a React + TypeScript frontend provides the UI. Credentials never leave your machine.

Why this exists:

- **Speed** — Batch-upload tests from ZIP/folders, paste solutions, set groups/points inline.
- **Safety** — API signing is handled server-side; no secrets in the browser.
- **Completeness** — Every Polygon API method is implemented (30+ endpoints).
- **Workflow** — An 8-step Upload Wizard guides you from problem creation to commit.

2. Architecture

High-Level Data Flow

```
Browser (React) --HTTP--> FastAPI Proxy (:8000) --HTTPS--> Polygon API
port 5173 signs requests polygon.codeforces.com
with SHA-512
```

The frontend never talks to Polygon directly. Every API call goes through the FastAPI proxy, which attaches the `apiKey`, `time`, and `apiSig` (SHA-512 HMAC) before forwarding.

Backend (Python)

- **FastAPI** — async web framework, handles CORS, form parsing, file uploads.
- **polygon_api.py** — core signing module. Builds the signature string as `rand/method?sorted_params#secret`, hashes with SHA-512.
- **config.json** — stores API key and secret locally (never committed to git).
- **httpx** — async HTTP client for non-file requests.
- **requests** — sync HTTP client for multipart file uploads (file content is included in signature).

Frontend (TypeScript)

- **React 18** with functional components and hooks.
- **Vite** — fast dev server with HMR at port 5173.
- **Tailwind CSS** — utility-first styling with custom 'Sunset Ember' theme.

- **React Router v6** — SPA routing (Problems list, Problem detail, Settings).
- **Lucide React** — icon library.
- **JSZip** — client-side ZIP parsing for batch test upload.

3. Getting Started

Prerequisites

- Python 3.10+ with pip
- Node.js 18+ with npm
- A Polygon API key and secret (from polygon.codeforces.com → API keys)

Setup

Backend:

```
cd backend
python -m venv venv
venv\Scripts\activate # Windows
# source venv/bin/activate # macOS/Linux
pip install -r requirements.txt
```

Frontend:

```
cd frontend
npm install
```

Running

Use the provided batch files, or start each service manually:

```
# Terminal 1 &mdash; Backend
cd backend && uvicorn main:app --reload --port 8000

# Terminal 2 &mdash; Frontend
cd frontend && npm run dev
```

Or simply run `start.bat` (Windows) to launch both.

First Use

- Open `http://localhost:5173` in your browser.
- Go to **Settings** and enter your Polygon API key, secret, and username.
- Return to **Problems** — your problem list loads automatically.
- Click any problem to manage it, or use the **Upload Wizard** for a guided flow.

4. Backend Deep Dive

API Signature (polygon_api.py)

Polygon requires every request to be signed. The signature is computed as:

```
hash_input = f"{rand_6_chars}/{method}?{sorted_key=value_pairs}#{secret}"
apiSig = rand_6_chars + sha512(hash_input.encode('utf-8')).hexdigest()
```

Critical: For file uploads, the file content (decoded as UTF-8) is included in the params dict *before* computing the signature. All params are then sent via `requests.post(url, files=all_parts)` — matching the approach used by the official `polygon-cli` tool.

Proxy Pattern (main.py)

Every endpoint in `main.py` follows the same pattern:

```
@app.post("/api/problem.saveSolution")
async def problem_save_solution(problemId: int = Form(...), ...):
    params = {"problemId": problemId, ...}
    content = await file.read()
    return await proxy(method, params, {"file": (name, content, mime)})
```

The `proxy()` function loads credentials, calls `call_polygon()`, and returns the raw Polygon response. Structured logging with timestamps and emoji indicators makes debugging easy.

Logging Format

```
=> [14:23:01] problem.saveSolution
params: problemId=523063, name=sol.cpp | files: file (sol.cpp)
OK [14:23:02] problem.saveSolution -> OK
```

5. Frontend Deep Dive

Routing & Pages

Route	Component	Description
/	ProblemsPage	Problem list with search, filter, create
/problem/:id	ProblemPage	Problem detail with 10 tabbed sections
/settings	SettingsPage	API credentials configuration
/wizard	UploadWizard	8-step guided problem upload

ProblemPage Tabs

Tab	File	Key Features
Info	InfoTab.tsx	Time/memory limits, I/O files, interactive toggle
Statement	StatementTab.tsx	LaTeX editor, Parse/Convert tool, View HTML preview
Checker/Validator	CheckerValidatorTab.tsx	Standard checkers dropdown, custom file upload
Solutions	SolutionsTab.tsx	Upload/paste/view/delete, 12 tag types
Tests	TestsTab.tsx	Add/edit/ZIP upload, inline group/points, bulk ops
Files	FilesTab.tsx	Source/resource/aux file management
Script	ScriptTab.tsx	Generator script editor
Packages	PackagesTab.tsx	Build & download packages
Tutorial	TutorialTab.tsx	General description & tutorial editor

API Client (api/client.ts)

A typed client module wraps all backend endpoints. It uses `fetch()` with three helpers: `get()` for query-string requests, `post()` for JSON bodies, and `postForm()` for multipart file uploads. All responses go through `handleResponse()` which parses JSON, detects Polygon FAILED status, and throws typed `ApiError` exceptions.

UI Component Library

Reusable components live in `components/ui/`:

- **Button** — primary/secondary/ghost/danger variants, loading state, icon support.
- **Card** — titled container with optional action buttons in header.
- **Modal** — animated overlay with customizable footer.

- **Input / Select / Textarea** — styled form controls with labels and helpers.
- **Badge** — status indicators, solution tag badges (MA, OK, RJ, etc.).
- **ToastContainer** — success/error/warning notifications.
- **Tabs** — horizontal tab navigation with lazy-mounting.

6. Upload Wizard (8-Step Flow)

The Upload Wizard (`wizard/UploadWizard.tsx`) guides users through creating and fully configuring a problem in one session:

Step	Description
1. Select/Create	Choose an existing problem or create a new one by name.
2. Problem Info	Set time limit, memory limit, I/O files, interactive flag.
3. Statement	Fill in all LaTeX sections: legend, input, output, scoring, notes.
4. Checker	Select a standard checker (wcmp, fcmp, etc.) or upload a custom .cpp file.
5. Validator	Upload a custom validator (optional).
6. Solutions	Upload solution files with tags (MA, OK, RJ, TL, WA, etc.).
7. Tests	Batch upload from ZIP with auto-parsed groups and indices.
8. Review & Commit	Review all settings and commit changes to Polygon.

7. Polygon API Coverage

All Polygon API methods are implemented in the backend proxy. Here's the complete mapping:

Category	Methods
Problems	problems.list, problem.create
Problem Config	problem.info, updateInfo, commitChanges, updateWorkingCopy, discardWorkingCopy
Statements	problem.statements, saveStatement, statementResources, saveStatementResource
Files	problem.files, saveFile, viewFile
Solutions	problem.solutions, saveSolution, viewSolution, editSolutionExtraTags
Tests	problem.tests, saveTest, testInput, testAnswer, setTestGroup, enableGroups, enablePoints
Test Groups	problem.viewTestGroup, saveTestGroup
Checker/Validator	problem.checker, validator, interactor, setChecker, setValidator, setInteractor
CV Tests	problem.validatorTests, checkerTests, saveValidatorTest, saveCheckerTest
Script	problem.script, saveScript

Category	Methods
Tags	problem.viewTags, saveTags
Tutorial	problem.viewGeneralDescription, viewGeneralTutorial, saveGeneralDescription, saveGeneralTutorial
Packages	problem.packages, buildPackage, package (download)
Contest	contest.problems

8. Key Features & Implementation Notes

ZIP Test Upload with Auto-Parsing

Tests can be uploaded from ZIP files, individual files, or folders. The parser:

- Extracts group numbers from filenames: `input_s2_003.txt` → group 2
- Extracts test indices: `test-001.txt` → index 1
- Skips answer/output files automatically
- Re-numbers tests sequentially starting after existing tests
- Auto-sets `useInStatements=true` for group 0 (sample tests)
- Detects duplicate test content and reports skipped tests with a warning toast

Inline Test Editing

Group and points columns in the test table support click-to-edit. Changes are saved to Polygon immediately and update local state without a full reload.

Test Groups & Subtask Management

- Groups and points are **auto-enabled** on Polygon when the Tests tab loads.
- Points policy is auto-corrected to **COMPLETE_GROUP** when loading groups.
- **Derive Dependencies** — parses the Scoring section of the statement to auto-fill group dependencies from a LaTeX tabular or plain text format.
- **Derive Points** — parses the Scoring section to extract group → points mapping and sets points on the first test of each group.

Statement HTML Preview

The **View HTML** button in the Statement tab converts the LaTeX statement into a styled HTML page with MathJax rendering, opens it in a new tab. If sample tests (group 0) exist, they are included. If a main correct solution (MA) is uploaded, sample outputs are fetched and displayed alongside inputs.

File Upload Signing

File uploads (solutions, checker, images) include file content in the SHA-512 signature hash and send all parameters via `files=` in the multipart request. This matches the `polygon-cli` approach and is *required* for Polygon to accept the signature.

Solution Management

- Upload via file picker or paste code directly with filename.
- 12 tag types matching Polygon's dropdown: MA, OK, RJ, TL, TO, TM, WA, PE, ML, RE, NR, FL.
- View solution source in a modal.
- Delete by overwriting with an empty file (Polygon has no native delete endpoint).

- 11 standard checkers matching Polygon: fcmp, hcmp, lcmp, ncmp, nyesno, rcmp4/6/9, wcmp, yesno.

9. Theme & Design System

The app uses a custom **Sunset Ember** dark theme with warm tones:

Token	Usage
Background (#110f0d)	Main page background
Surface (#1e1b17)	Card and panel backgrounds
Deep surface (#1a1714)	Input fields, dropdowns
Border (#362f28)	Dividers, outlines
Accent (#f59e0b)	Primary amber accent for actions, highlights
Accent hover (#fbbf24)	Lighter amber for hover states
Accent muted (#4a3520)	Subtle amber backgrounds

Typography & Animations

- **Inter** for UI text, **JetBrains Mono** for code and monospace fields.
- CSS animations: `fadeInUp`, `fadeIn`, `scaleIn`, `slideInLeft`, `slideUp`, `stagger-children`.
- Active button scale effect: `active:scale-[0.97]`.
- Amber scrollbar and focus ring styling.

10. File Structure Reference

Backend

```
backend/  
main.py # FastAPI app, all proxy endpoints, logging  
polygon_api.py # SHA-512 signing, call_polygon()  
config.json # API key & secret (gitignored)  
requirements.txt # Python dependencies
```

Frontend

```
frontend/src/  
App.tsx # Router setup  
api/client.ts # Typed API client (get/post/postForm)  
context/AppContext.tsx # Global state (problems, toasts)  
types/polygon.ts # TypeScript interfaces  
pages/  
ProblemsPage.tsx # Problem list + search  
ProblemPage.tsx # Problem detail (10 tabs)
```

```
SettingsPage.tsx # Credentials config
tabs/
InfoTab.tsx, StatementTab.tsx, CheckerValidatorTab.tsx,
SolutionsTab.tsx, TestsTab.tsx, FilesTab.tsx,
ScriptTab.tsx, PackagesTab.tsx, TutorialTab.tsx
wizard/
UploadWizard.tsx # 8-step guided upload
components/
Sidebar.tsx # Navigation sidebar
ui/ # Button, Card, Modal, Input, Badge, etc.
```

Startup Scripts

- `start.bat` — launches both backend and frontend.
- `start_backend.bat` — backend only.
- `start_frontend.bat` — frontend only.